Makefiles: Beginner to Expert

A complete study guide to GNU Make — from your first rule to advanced techniques used in production build systems.

Part 1: Beginner

1.1 What is Make?

Make is a build automation tool. It reads a file called `Makefile` and decides **what to rebuild** based on **file timestamps**. If a source file is newer than its output, Make rebuilds the output; otherwise it skips it.

Why use it?

- Automates repetitive commands (compile, test, deploy, clean)
- Only rebuilds what changed (incremental builds)
- Self-documenting: the Makefile *is* the build recipe

Run it with:

```
make            # runs the first target in the Makefile
make target     # runs a specific target
make -f my.mk   # use a different file name
```

1.2 Anatomy of a rule

```
target: prerequisites
    recipe
```

- **target** — the file to build (or an action name)
- **prerequisites** — files the target depends on
- **recipe** — shell commands to build the target

⚠ **The recipe MUST be indented with a TAB, not spaces.** This is the #1 beginner error. The error looks like: `*** missing separator. Stop.`

Example:

```
hello: hello.c
    gcc -o hello hello.c
```

Meaning: "`hello` depends on `hello.c`. If `hello.c` is newer than `hello` (or `hello` doesn't exist), run `gcc`."

1.3 Multiple rules and chaining

```
app: main.o utils.o
    gcc -o app main.o utils.o

main.o: main.c
    gcc -c main.c

utils.o: utils.c
    gcc -c utils.c
```

Make builds a **dependency graph**. Asking for `app` causes Make to first ensure `main.o` and `utils.o` are up to date.

1.4 Phony targets

Some targets are actions, not files:

```
.PHONY: clean all

all: app

clean:
    rm -f app *.o
```

`.PHONY` tells Make: "don't look for a file named `clean`; always run the recipe." Without it, a file literally named `clean` in your directory would break `make clean` ("`clean` is up to date").

Common phony targets: `all`, `clean`, `install`, `test`, `run`, `help`.

1.5 Variables

```
CC = gcc
CFLAGS = -Wall -g
TARGET = app

$(TARGET): main.o utils.o
    $(CC) $(CFLAGS) -o $(TARGET) main.o utils.o
```

- Reference with `$(VAR)` or `${VAR}`
- Single-character variables can skip parens: `$X`
- Override from the command line: `make CC=clang`

1.6 Comments and line continuation

```
# This is a comment
SOURCES = main.c \
          utils.c \
          parser.c
```

Beginner checklist

- [ ] Tabs for recipes, never spaces
- [ ] First target = default target
- [ ] Mark action targets `.PHONY`
- [ ] Use variables for compiler, flags, filenames

Part 2: Intermediate

2.1 Automatic variables

These are set by Make per-rule and eliminate repetition:

Variable	Meaning
`$@`	The target name
`$<`	The **first** prerequisite
`$^`	**All** prerequisites (deduplicated)
`$+`	All prerequisites (with duplicates)
`$?`	Prerequisites **newer** than the target
`$*`	The stem matched by `%` in a pattern rule
`$(@D)` / `$(@F)`	Directory / file part of the target

Rewriting the earlier example:

```
app: main.o utils.o
    $(CC) $(CFLAGS) -o $@ $^

main.o: main.c
    $(CC) $(CFLAGS) -c $< -o $@
```

2.2 Pattern rules

One rule to compile *any* `.c` into a `.o`:

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

`%` is a wildcard that must match the same stem on both sides. Now you never write per-file compile rules again.

2.3 Wildcards and file lists

```
SRCS := $(wildcard src/*.c)
OBJS := $(SRCS:.c=.o)          # substitution reference
# or equivalently:
OBJS := $(patsubst %.c,%.o,$(SRCS))

app: $(OBJS)
    $(CC) -o $@ $^
```

⚠ `*.c` in a prerequisite list works, but in a variable assignment you must use `$(wildcard ...)` — a bare `*` is not expanded there.

2.4 `=` vs `:=` vs `?=` vs `+=`

Operator	Name	Behavior
`=`	Recursive	Expanded **every time it's used** (lazy)
`:=`	Simple	Expanded **once, at definition** (eager)
`?=`	Conditional	Set only if not already defined
`+=`	Append	Adds to existing value

```
X = $(Y)      # X reflects Y's value at use time
Y := hello
A := $(B)     # A is empty forever — B wasn't defined yet
B := world
```

Rule of thumb: prefer `:=` — it's predictable and faster. Use `=` only when you *need* lazy evaluation. Use `?=` for user-overridable defaults.

2.5 Useful built-in functions

```
$(subst from,to,text)          # literal substitution
$(patsubst %.c,%.o,$(SRCS))    # pattern substitution
$(filter %.c %.h,$(FILES))     # keep matching words
$(filter-out test%,$(FILES))   # remove matching words
$(sort $(WORDS))               # sort + deduplicate
$(word 2,$(LIST))              # nth word
$(words $(LIST))               # count
$(dir src/foo.c)               # -> src/
$(notdir src/foo.c)            # -> foo.c
```

```
$(basename src/foo.c)          # -> src/foo
$(suffix src/foo.c)            # -> .c
$(addprefix build/,$(OBJS))    # prepend to each word
$(addsuffix .bak,$(FILES))     # append to each word
$(shell date +%Y%m%d)          # run a shell command
$(foreach v,$(LIST),$(v).o)    # loop
$(if $(COND),then,else)        # conditional expression
$(error message)               # abort with error
$(warning message)             # print warning, continue
$(info message)                # print info
```

2.6 Conditionals

```
DEBUG ?= 0

ifeq ($(DEBUG),1)
    CFLAGS += -g -O0
else
    CFLAGS += -O2
endif

ifdef VERBOSE
    Q =
else
    Q = @
endif
```

Note: conditionals (`ifeq`, `ifdef`, etc.) are evaluated when the Makefile is **read**, not when recipes run. They must NOT be indented with a tab (or they become part of a recipe).

2.7 Recipe prefixes

```
install:
    @echo "Installing..."      # @ = don't echo the command
    -rm -f /tmp/old.lock       # - = ignore errors from this line
    +$(MAKE) -C subdir         # + = run even under 'make -n'
```

2.8 Each recipe line is a separate shell!

```
bad:
    cd build
    ls          # runs in the ORIGINAL directory — cd had no effect!

good:
    cd build && ls
```

```
also_good:
    cd build; \
    ls
```

Use `.ONESHELL:` (GNU Make ≥ 3.82) to run the whole recipe in one shell.

2.9 Directories as prerequisites (order-only)

Timestamps of directories change when files inside change — using them as normal prerequisites causes spurious rebuilds. Use **order-only prerequisites** (after a `|`):

```
build/%.o: src/%.c | build
    $(CC) -c $< -o $@

build:
    mkdir -p $@
```

`build` must *exist* before compiling, but its timestamp never triggers a rebuild.

2.10 A solid intermediate Makefile template

```
CC      := gcc
CFLAGS  := -Wall -Wextra -O2
LDFLAGS :=
SRC_DIR := src
OBJ_DIR := build
TARGET  := app

SRCS := $(wildcard $(SRC_DIR)/*.c)
OBJS := $(patsubst $(SRC_DIR)/%.c,$(OBJ_DIR)/%.o,$(SRCS))

.PHONY: all clean run

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $^

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c | $(OBJ_DIR)
    $(CC) $(CFLAGS) -c $< -o $@

$(OBJ_DIR):
    mkdir -p $@

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -rf $(OBJ_DIR) $(TARGET)
```

Part 3: Advanced

3.1 Automatic header dependency tracking

The classic problem: editing `foo.h` doesn't rebuild `foo.o` unless you list every header manually. Solution: let the compiler generate dependency files.

```
DEPFLAGS = -MMD -MP
CFLAGS  += $(DEPFLAGS)

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c | $(OBJ_DIR)
    $(CC) $(CFLAGS) -c $< -o $@

-include $(OBJS:.o=.d)
```

- `-MMD` writes a `.d` file containing `foo.o: foo.c foo.h ...`
- `-MP` adds phony targets for headers (prevents errors when headers are deleted)
- `-include` includes the `.d` files, silently ignoring missing ones

This is the single most important technique for correct C/C++ Makefiles.

3.2 Parallel builds

```
make -j8        # 8 jobs in parallel
make -j$(nproc) # one job per CPU core
```

Your Makefile must have **correct dependencies** for `-j` to be safe. Common parallel bugs:

- Two rules writing the same file
- Missing dependencies that "happened to work" serially
- Recipes with multiple outputs declared wrong (see grouped targets below)

3.3 Grouped targets (GNU Make ≥ 4.3)

When one recipe produces multiple files:

```
# WRONG under -j: Make thinks the recipe runs once PER target
parser.c parser.h: parser.y
    bison -d $<

# RIGHT: &: declares one recipe producing both files
parser.c parser.h &: parser.y
    bison -d $<
```

On older Make, the traditional workaround is a pattern rule (pattern rules with the same stem *are* treated as grouped) or an intermediate stamp file.

3.4 Double-expansion: `.SECONDEXPANSION`

Allows automatic variables inside prerequisite lists:

```
.SECONDEXPANSION:
$(BINS): %: $$(OBJ_DIR)/%.o
    $(CC) -o $@ $<
```

Escaped `$$` variables are expanded a second time, after `$@`/`$*` are known.

3.5 `define`, `call`, and `eval` — Make metaprogramming

Generate rules programmatically:

```
MODULES := core net ui

# A "template" taking one parameter: $(1)
define MODULE_template
$(1)_SRCS := $$(wildcard $(1)/*.c)
$(1)_OBJS := $$($(1)_SRCS:.c=.o)
lib$(1).a: $$($(1)_OBJS)
    ar rcs $$@ $$^
endef

# Instantiate the template for each module
$(foreach mod,$(MODULES),$(eval $(call MODULE_template,$(mod))))
```

- `define ... endef` creates a multi-line variable
- `$(call TEMPLATE,arg1,arg2)` substitutes `$(1)`, `$(2)`, ...
- `$(eval ...)` parses the result **as Makefile syntax**
- Inside a template destined for `eval`, use `$$` for anything that should be expanded late

This is how large projects avoid writing hundreds of near-identical rules.

3.6 Target-specific and pattern-specific variables

```
debug: CFLAGS += -g -O0 -DDEBUG
debug: app

release: CFLAGS += -O3 -DNDEBUG
release: app

test/%.o: CFLAGS += -I test/include
```

The variable applies to the target **and all its prerequisites** built on its behalf.

3.7 VPATH and vpath

Tell Make where to search for prerequisites:

```
VPATH = src:include          # search these dirs for all files
vpath %.c src                # search src/ only for .c files
vpath %.h include
```

Useful for out-of-tree builds, though explicit paths are usually clearer.

3.8 Recursive vs non-recursive Make

Recursive (`$(MAKE) -C subdir`) is simple but flawed: each sub-make sees only a fragment of the dependency graph, breaking parallelism and correctness ("Recursive Make Considered Harmful", Peter Miller, 1998).

Non-recursive: one top-level Makefile `include`s fragments:

```
# Top-level Makefile
include src/module.mk
include net/module.mk

# src/module.mk
SRCS += src/a.c src/b.c
```

One graph, perfect parallelism, correct incremental builds. More setup effort, usually worth it for big projects.

When you *do* recurse, always use `$(MAKE)` (never literal `make`) so flags and the jobserver are inherited.

3.9 Special targets worth knowing

Target	Effect
`.PHONY`	Targets that aren't files
`.DEFAULT_GOAL := all`	Set default target explicitly
`.ONESHELL:`	Run whole recipe in one shell
`.DELETE_ON_ERROR:`	Delete target if recipe fails (**always use this!**)
`.SECONDARY:`	Don't delete intermediate files
`.PRECIOUS: %.o`	Keep files even on interrupt/error
`.INTERMEDIATE:`	Mark files as intermediate (auto-deleted)
`.NOTPARALLEL:`	Disable parallelism for this Makefile

Target	Effect
`.EXPORT_ALL_VARIABLES:`	Export all vars to sub-makes
`.SILENT:`	Like `@` on every line

`.DELETE_ON_ERROR:` deserves emphasis: without it, a recipe that fails halfway leaves a corrupt, *newer-than-sources* output file, and the next `make` skips it. Put it in every Makefile.

3.10 Make flags you should know

```
make -n          # dry run: print commands without running
make -B          # unconditionally rebuild everything
make -k          # keep going after errors
make -j N        # parallel jobs
make -C dir      # change directory first
make -p          # print the entire database (rules + variables)
make --debug=b   # basic debug: why is each target rebuilt?
make V=1         # (convention) verbose output
make --trace     # show recipes and why they run (Make ≥ 4.0)
make -O          # synchronize output of parallel jobs (Make ≥ 4.0)
```

Debugging one-liner — print any variable's value:

```
print-%:
    @echo '$*=$($*)'
```

```
make print-CFLAGS    # -> CFLAGS=-Wall -Wextra -O2
```

Part 4: Expert

4.1 How Make actually works (the two phases)

1. **Read phase**: Make parses the Makefile, expands all *immediate* constructs (`:=`, `ifeq`, `include`, rule *targets and prerequisites*), and builds the dependency graph.
2. **Target-update phase**: Make walks the graph, and only now expands *deferred* constructs (recipes, `=` variables).

Understanding which expansions happen in which phase explains almost every "why is my variable empty?" mystery:

- Targets & prerequisites: expanded **immediately**
- Recipes: expanded **at execution time**
- `=` right-hand side: deferred; `:=` right-hand side: immediate

4.2 Empty targets / stamp files

Track actions that don't naturally produce a file:

```
.deployed: $(TARGET)
    ./deploy.sh $(TARGET)
    touch $@
```

The empty `.deployed` file records *when* deployment last happened; redeploys occur only when `$(TARGET)` changes.

4.3 Rebuilding when the *command* changes

Make only compares timestamps of files — change `CFLAGS` and nothing rebuilds. Expert fix: hash the flags into a stamp file dependency:

```
COMPILE_CMD = $(CC) $(CFLAGS)

.flags: FORCE
    @echo '$(COMPILE_CMD)' | cmp -s - $@ || echo '$(COMPILE_CMD)' > $@

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c .flags | $(OBJ_DIR)
    $(COMPILE_CMD) -c $< -o $@

.PHONY: FORCE
FORCE:
```

`.flags` is rewritten only when the command line actually differs, so objects rebuild exactly when flags change.

4.4 The jobserver

With `-jN`, the top-level Make creates a **jobserver** (a token pipe). Sub-makes and jobserver-aware tools acquire tokens before starting work, so total parallelism across the whole tree never exceeds N. This is why you must:

- Use `$(MAKE)`, not `make`, for recursion
- Prefix recursive calls with `+` if the recipe line isn't obviously a make invocation

4.5 Order of variable precedence (highest → lowest)

1. `override` directives in the Makefile
2. Command-line assignments (`make CC=clang`)
3. Makefile assignments
4. Environment variables (with `-e`, these beat Makefile assignments)
5. Built-in defaults (`CC = cc`, etc.)

```
override CFLAGS += -Wall   # appends even against command-line CFLAGS
```

4.6 Secondary expansion vs eval vs constructed includes

Three escalating metaprogramming tools:

1. **Target/pattern-specific vars** — per-target tweaks. Simple.
2. `.SECONDEXPANSION` — computed prerequisites. Medium.
3. `$(eval $(call ...))` — generate whole rule sets. Heavy but unlimited.
4. **Generated + included Makefiles** — write `.mk` fragments with a script, `include` them. Make automatically **re-executes itself** if an included file was rebuilt by a rule — this is the mechanism behind `.d` dependency files.

That last point is subtle and powerful: if any `include`d file is out of date and a rule exists to rebuild it, Make rebuilds it and **restarts from scratch**.

4.7 Sanity/robustness header for production Makefiles

```
SHELL := bash
.SHELLFLAGS := -eu -o pipefail -c   # fail fast in recipes
.ONESHELL:                          # whole recipe = one shell
.DELETE_ON_ERROR:                   # no corrupt outputs
MAKEFLAGS += --warn-undefined-variables
MAKEFLAGS += --no-builtin-rules     # explicit is better than implicit
.DEFAULT_GOAL := help
```

(Widely known as the "strict mode" preamble.)

4.8 Self-documenting help target

```
.PHONY: help
help:  ## Show this help
    @grep -E '^[a-zA-Z_-]+:.*?## ' $(MAKEFILE_LIST) | \
      awk 'BEGIN {FS = ":.*?## "}; {printf "\033[36m%-20s\033[0m %s\n", $$1, $$2}'

build: ## Build the application
    ...

test: ## Run the test suite
    ...
```

`make help` prints a colored command menu from the `##` comments.

4.9 Make for non-C workflows

Make is a general DAG executor — great for data pipelines, docs, Docker, deployment:

```
data/clean.csv: data/raw.csv scripts/clean.py
    python scripts/clean.py $< > $@

report.pdf: report.md data/clean.csv
    pandoc $< -o $@

.PHONY: docker-build
docker-build: .docker-stamp
.docker-stamp: Dockerfile $(SRCS)
    docker build -t myapp .
    touch $@
```

Any workflow with "outputs derived from inputs" benefits from Make's incrementality.

4.10 Known limitations & when to reach for other tools

Make's weak spots:

- Timestamps, not content hashes (touch a file → rebuild)
- Whitespace/filenames with spaces are essentially unsupported
- No built-in sandboxing; undeclared deps go undetected
- Portability: GNU Make vs BSD make vs POSIX make differ significantly

Alternatives and when they win:

- **CMake / Meson** — C/C++ projects needing cross-platform config (they *generate* Make/Ninja files)
- **Ninja** — ultra-fast execution backend; not meant to be hand-written
- **Bazel / Buck2** — hermetic, content-hashed, remote-cached monorepo builds
- **just** — Make-like command runner without the build-graph semantics (great when you only want phony targets)

Knowing Make deeply still pays off: its model (declarative DAG + incrementality) underlies all of them.

Quick reference card

```
# Rule:            target: prereqs        (TAB) recipe
# Auto vars:       $@ target   $< first prereq   $^ all prereqs   $* stem   $?
newer prereqs
# Assignment:      := eager    = lazy    ?= default    += append
# Pattern rule:    %.o: %.c
# Order-only:      target: normal | order-only
# Grouped:         a b &: input           (Make >= 4.3)
# Include deps:    -include $(OBJS:.o=.d)      with CFLAGS += -MMD -MP
# Must-haves:      .PHONY  .DELETE_ON_ERROR  mkdir via order-only prereq
# Debug:           make -n / -p / --debug=b / --trace ; print-% rule
# Parallel:        make -j$(nproc) ; use $(MAKE) for recursion
```

Suggested learning path

1. Write a 3-file C project Makefile by hand (Parts 1–2)
2. Add auto-dependencies (`-MMD -MP`) and out-of-tree `build/` dir
3. Break it with `-j8`, then fix the dependencies until it's parallel-safe
4. Add `debug`/`release` via target-specific variables
5. Read your distro's largest hand-written Makefile (e.g., the Linux kernel's) with `make -p` and `--trace` as guides
6. Read the GNU Make manual cover to cover — it's short and excellent: https://www.gnu.org/software/make/manual/